



Lecture 3: The Anatomy of a Svelte Component

INTERVIEW QUESTION

[> CORE](#)

What are the three parts of a `.svelte` file, and what is each one for?

You open a `.svelte` file for the first time and see script, markup, and style all in one place. How do these three pieces fit together?

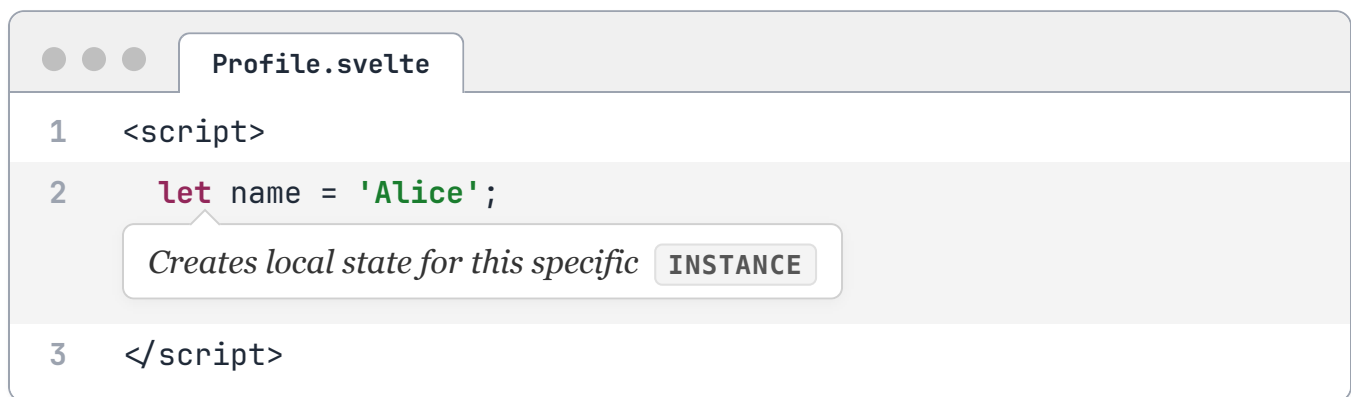
A Svelte component is a self-contained building block for your user interface. It is written in a `.svelte` file that uses a superset of HTML. Inside this single file, you can define three distinct sections. All three sections are entirely optional—you only include what you need.

1. The `<script>` Block (Logic)

The `<script>` block contains the JavaScript (or TypeScript) that controls how your component behaves.

When we define a user profile card in our code, we usually want to show more than one user on the screen at once. Should we copy and paste the profile card code three times? Or do we write it once as a blueprint, and then tell the framework "build me three of these"? This is a decision about how we reuse code. We need a way to stamp out multiple independent copies of the same UI blueprint, where each copy holds its own independent state (like its own name or counter variable). Each independent copy built from the blueprint is called a **component instance**.

The code inside the `<script>` block runs every time a new component instance is created. Any variable you declare here becomes the private, local state for that specific copy.



The `<script module>` Alternative

We know each copy of a component keeps its own private data. But what if all the copies need to talk to each other?

Imagine you are building a webpage with five audio players on it (like a podcast list). When a user clicks "Play" on Player #3, Player #1 needs to pause. You don't want two podcasts playing over each other. But if we build an `AudioPlayer.svelte` component and stamp out 5 copies, we have 5 independent "islands." Player #1 has no idea that Player #3 even exists.

So how do we make the 5 islands talk to each other? Do we pass messages all the way up to the main webpage, and then back down to all the other players telling them to stop? That's exhausting. Do we create a completely separate JavaScript file just to hold one variable called `currentlyPlaying` ? That feels like overkill.

This is a decision about data architecture. We need a way for these 5 component instances to share a single, *communal brain*. We want exactly one variable that all 5 players can look at and update. We need a place inside our `.svelte` file to run code exactly *once*, when the file is first loaded, rather than every time a new player is stamped out.

This shared, run-once area is called the **module script** (written as `<script module>`).

● ● ● **AudioPlayer.svelte**

```
1  <script module>
2    let currentlyPlaying = null;
3  </script>
4
5  <script>
6    let audioSrc = 'podcast.mp3';
7
8    function play() {
9      currentlyPlaying = audioSrc;
10    }
11  </script>
```

The communal brain: shared across ALL 5 players

The local island: runs for every NEW player created

Player 3 updates the shared brain!

Because the module script only runs once, any variable you put inside it is shared across every single copy of that component on the screen.

2. The Markup (Structure)

The markup is the HTML structure of your component. It sits outside of any tag, directly in the body of the `.svelte` file. You can freely reference any variable declared in your `<script>` block inside your markup.

KEY TAKEAWAY

A Svelte file is just HTML with superpowers, not JavaScript with HTML crammed inside it.

If you look at a React component, you are fundamentally writing a JavaScript function that happens to return HTML (via JSX). In Svelte, you are fundamentally writing an HTML file that happens to include logic. This inversion means that any valid HTML file is already a valid Svelte component.

3. The `<style>` Block (Aesthetics)

The `<style>` block contains the CSS for your component.

When we write CSS for our components, we need a way to style elements like a `<p>` tag. Should we just put `p { color: red; }` in our CSS? But what if another component also has a `<p>` tag? Will they all turn red, leaking styles everywhere? Do we need to invent complex class names like `class="my-unique-component-p-tag"` for every single element?

This is a decision about the architecture of our CSS. We need a way to guarantee that the styles written inside a file only affect the elements in that exact file, and nowhere else in the app. This guaranteed isolation is called **scoped styling**. Scoped styling ensures that your CSS cannot accidentally leak out and break other parts of your application.

```
Profile.svelte
1  <style>
2    p {
3      color: burlywood; /* ✓ safely **SCOPED** to this component only */
4    }
5  </style>
```

INTERVIEW TIP

To impress the interviewer: When asked about `.svelte` files, emphasize that scoping is handled automatically by the compiler. You don't need external tooling like CSS Modules or BEM naming conventions; Svelte simply hashes the classes for you.

Summary

A `.svelte` file consists of three optional parts. The `<script>` block holds the JavaScript logic that runs when a component instance is created, managing its local state. The HTML markup defines the structure of the component and can reference variables from the script. Finally, the `<style>` block contains CSS that is automatically scoped to affect only the markup within that specific file, preventing global style leaks.

	SVELTE COMPONENT (Single-File Anatomy)	REACT COMPONENT (JSX Anatomy)
Foundation	Superset of HTML	JavaScript function
Styling	Built-in, scoped <code><style></code> tags	Requires external CSS-in-JS or CSS Modules
Logic	<code><script></code> block	Function body